

Simulation Study of Active Particle Systems

Author: Michel Faloughi

Supervisor: Prof Alexandre Stauffer

Module Code: 6CCM345A

February 2025

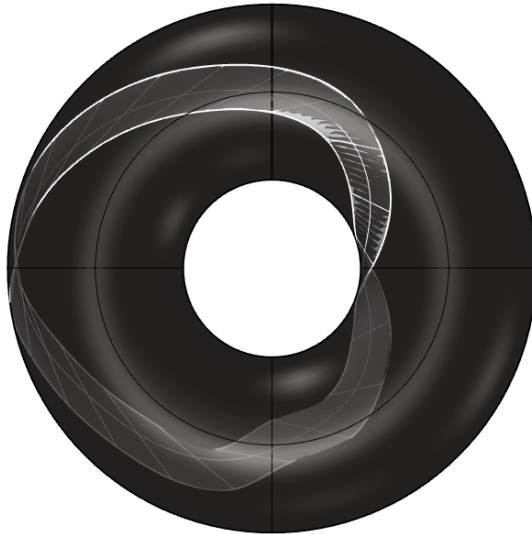


Abstract

Self-organizing particle systems are fascinating models that capture how simple local interactions can lead to large-scale collective behavior. From biological swarms to traffic flow, the ability of individual units to form clusters without central control is an important phenomenon with many real-world applications.

This study explores how active particles move and interact in a structured environment. We examine how different parameters influence clustering behavior, investigating factors such as movement persistence, interaction strength, and initial configurations. To quantify the system's evolution, we analyze statistical distributions of clustering times and densities, employing mathematical modeling to uncover underlying patterns.

Through extensive simulations, we identify trends in cluster formation and metastable states, offering insights into how systems transition from disorder to order. While this research is motivated by theoretical curiosity, it also has potential applications in understanding active matter, hydrodynamics, and traffic systems.



Contents

1	Introduction	5
2	Model Setup	6
2.1	Two-Dimensional Particle System	6
2.1.1	Initialization	6
2.1.2	Update Rules	7
2.2	One-Dimensional Particle System	7
2.2.1	Initialization	7
2.2.2	Update Rules	8
2.2.3	Visualization	8
2.3	Simulation Software Documentation	9
2.3.1	Running the Simulation	9
2.3.2	User Controls	11
3	Visual Analysis	12
3.1	Example 2D rendered simulation	12
3.2	Example 1D Rendered Simulation	18
4	Statistical Analysis	20
4.1	Parameter-Wise Impact Analysis	20
4.1.1	Baseline Parameters Analysis	21
4.1.2	Other Settings of Parameters: α	26
4.1.3	μ	27
4.1.4	δ	29
4.1.5	ϵ	31
4.2	Evolution of Total Clusters	33
4.2.1	200×200 Particle Systems	34

4.2.2	100×100 Particle Systems	36
5	Conclusion	38

1 Introduction

Clustering behavior is a fundamental aspect of self-organizing systems, appearing in nature, physics, and engineered systems. Whether it is the movement of biological cells, the flow of traffic, or the dynamics of granular materials, the spontaneous formation of dense clusters emerges from simple interaction rules. In particular, active particles (individual agents that move with self-propulsion) exhibit clustering phenomena that have drawn increasing attention in recent years.

One of the most intriguing aspects of active particle systems is their ability to undergo *motility-induced phase separation* (MIPS). Unlike traditional phase separation in equilibrium systems, MIPS occurs purely due to differences in particle motion, even in the absence of attractive forces. Particles slow down when they encounter dense regions, reinforcing local aggregation and leading to spontaneous clustering. While this phenomenon has been widely observed in simulations, there is still much to uncover about the precise mechanisms governing its onset and evolution.

This project investigates how different movement and interaction parameters influence clustering in an active particle system. We analyze how clusters form, merge, and stabilize over time, using both visual and statistical approaches. In particular, we aim to answer the following key questions:

1. How do different parameter settings impact the speed and nature of clustering?
2. What mathematical distributions best describe clustering times and densities?
3. Do metastable configurations play a role in delaying full clustering?

A major component of this project is the development of a simulation framework capable of modeling and tracking clustering behavior under various conditions. The complexity of the system required significant computational effort, leading to software optimizations that made large-scale simulations feasible.

Despite the progress made, many questions remain open. Given the vast number of possible research directions, this study represents only a first step in a broader exploration of clustering phenomena. We also highlight potential real-world applications, including modeling traffic congestion, understanding biological self-organization, and analyzing active matter physics.

The structure of this paper is as follows. In Section 2, we introduce the particle system model, including its initialization, update rules, and simulation framework. Section 3 presents a visual analysis of how clusters form and evolve in both one-dimensional and two-dimensional settings. Section 4 delves into a statistical examination of clustering times and densities, fitting mathematical distributions to our data. Finally, in Section 5, we conclude with a discussion of modest findings, limitations, and possible future research directions.

2 Model Setup

2.1 Two-Dimensional Particle System

The simulation is implemented in Python mainly using `Pygame` and `NumPy`. It models a system of interacting particles on a discrete toroidal lattice (torus) of size $n \times m$. Each lattice site can either be occupied by a particle or remain empty. Visualization will follow in the next Section 3

2.1.1 Initialization

The system is initialized with the following parameters:

- $\mu \in (0, 1)$: Probability of spawning a particle in a given grid cell.
- $\delta \in [0, 1]$: Probability of a particle changing direction.
- $\alpha \in [0, 1]$: Probability of a particle being active.
- $\epsilon \in [0, 1]$: Probability affecting movement behavior.
- $n \geq 0$: Grid height (number of rows).
- $m \geq 0$: Grid width (number of columns).
- `direction_weights` : Probability distribution for movement in different directions.

Starting the simulation will create a unique identification number for that particular particle system which is automatically stored in a database along with the parameters of the system, number of particles, number of iterations, and other data in order to conduct some statistical analysis on the simulations later (see Section 4)

First, the code loops through every lattice site, and spawns a particle there with probability μ and leaves it blank with probability $1 - \mu$. Therefore the expected number of particles in the system is given by:

$$\mathbb{E}[\text{Total Particles}] = \mu \times n \times m. \quad (2.1)$$

Each spawned particle is assigned an initial position: (x, y) on the torus, a velocity vector v chosen uniformly at random from the set $V = \{(1, 0), (-1, 0), (0, 1), (0, -1)\}$ (respectively down, right, up, left) and an activity state.

Particles are $\begin{cases} \text{active,} & \text{with probability } \alpha, \\ \text{inactive,} & \text{with probability } 1 - \alpha \end{cases}$

2.1.2 Update Rules

At each iteration:

1. A particle is selected uniformly at random, i.e with expected probability $\mathbb{E}[\Pr[P = p]] = \frac{1}{\mu n m}$.
2. If the particle is *non-active*, it selects a random velocity vector from V at each step.
3. If the particle is *active*, it retains its velocity with probability $1 - \delta$ and selects a new velocity with probability δ .
4. Then, with probability ϵ , the particle moves in its assigned velocity direction. With probability $1 - \epsilon$, it moves in a random direction, i.e behaving like a non-active particle for that step.
5. A particle moves only if the target site is empty; otherwise, it remains in place, effectively causing particles to interactively move and restrict each other's movements

The movement of a particle at position (x, y) follows a discrete-time stochastic process:

$$(x', y') = (x + v_x, y + v_y) \mod n, \quad (2.2)$$

where $v = (v_x, v_y)$ is the assigned velocity. The modulo operator is employed here to 'wrap around' when a particle is at the edge of a screen, since we are dealing with a torus.

2.2 One-Dimensional Particle System

The one-dimensional model is a simplified version of the two-dimensional system, where particles move along a circular one-dimensional lattice of length N . The leftmost and rightmost positions are connected, forming a periodic topology.

2.2.1 Initialization

The system is initialized with:

- N : Number of discrete positions (lattice sites).
- $\mu \in (0, 1)$: Probability of a site being occupied at initialization.
- $\delta \in [0, 1]$: Probability of an active particle changing direction.
- $\alpha \in [0, 1]$: Probability of a particle being active.
- $\epsilon \in [0, 1]$: Probability affecting movement behavior.

Again, the code loops through every lattice site, and spawns a particle there with probability μ and leaves it blank with probability $1 - \mu$. Therefore the expected number of particles in the system for the one-dimensional case is:

$$\mathbb{E}[\text{Total Particles}] = \mu \times N. \quad (2.3)$$

Each spawned particle is assigned an initial position x in the range $[0, N - 1]$, a velocity direction v chosen uniformly at random from the set $\{-1, +1\}$ (left or right), and an activity state, where, as before, the particle is active with probability α and inactive with probability $1 - \alpha$.

2.2.2 Update Rules

At each iteration:

1. A particle is selected uniformly at random.
2. If the particle is *non-active*, it selects a new direction $v \in \{-1, +1\}$ at every step and moves accordingly.
3. If the particle is *active*, it retains its velocity with probability $1 - \delta$ or selects a new velocity with probability δ .
4. With probability ϵ , the particle moves in its assigned velocity direction. With probability $1 - \epsilon$, it selects a new direction and moves accordingly.
5. A particle moves only if the target site is unoccupied.

The movement of a particle at position x follows:

$$x' = (x + v) \mod N. \quad (2.4)$$

2.2.3 Visualization

The evolution of the system is represented as a spacetime plot, where the horizontal axis corresponds to particle positions, and the vertical axis represents time. The bottom row displays the most recent configuration of particle positions at all times, and at every `refresh_rate` iterations, a new row is appended to the spacetime graph.

Since directly visualizing a large number of particles in a single row is impractical on a screen, we aggregate the N particles into N/width bins. Each bin is assigned a color based on the density of particles it contains, reflecting the degree of clusteredness.

The clustering metric for each bin is computed as the ratio of occupied particle slots to the total available slots, producing a normalized value between 0 (completely empty) and 1 (completely

full). This clustering value is then mapped to a color map, as illustrated in Figure 2.1, where black represents an empty bin and white represents a fully occupied one.

For visualizations of our 1D Interactive Particle Systems simulations, refer to Section 3.2.

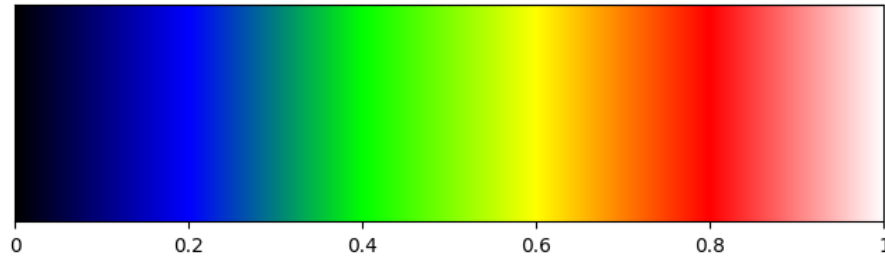


Figure 2.1: Colormap to show degree of clusteredness

2.3 Simulation Software Documentation

The simulation software is a multi-class object-oriented program, entirely implemented by myself and constitutes a big part of my project. It was designed to be versatile and adapt to any change of parameters/update rules/experiments and hopefully other computational researchers might use it to conduct interactive particle simulations on discrete lattices themselves. It's also very fun to play with. This section outlines how to run the program, interact with it, and adjust key parameters.

2.3.1 Running the Simulation

Assuming Python 3.8 and above is installed in the environment where the code will be run, to start the simulation, it's as easy as executing the following command in a terminal or command prompt:

```
python main.py          # For the 2D particle simulator
python OneDimensionalMain.py # For the 1D particle simulator
```

This launches a graphical window where the user can see the evolution of the particle system in real time. The `main` files contain the default parameters that we found to be the most interesting both visually and computationally. However, the files are easily customizable to adjust simulation preferences, calculations, and actions before or after the simulation.

The user can customize the simulation parameters, computations, and initial/terminal settings as follows:

- **Core Parameters:** As seen before, these include μ , δ , ϵ , α , as well as `width`, `height`, `direction_weights`.
- **Initial and Terminal Settings:**

- `init_paused_state` : Boolean flag indicating whether the simulation starts in a paused state, useful for inspecting the system’s initial configuration.
- `init_one_step_mode` : Boolean flag for enabling single-step execution at startup, allowing visualization of iteration-by-iteration changes.
- `init_refresh_rate` : Integer value specifying the initial display refresh rate (frames per second). This can be adjusted during the simulation (see Section 2.3.2).
- `num_iterations` : Total number of iterations before termination.
- `middle_cluster_size` : Initial size of the central particle cluster. Typically set to -1, as we prefer to start with particles uniformly distributed rather than a middle cluster.
- `is_rendering` : Boolean flag determining whether the simulation is visually rendered. If set to `False`, the simulation will run in the background without displaying the particle system in real time, useful for faster computational experiments.

• **Computational Methods:**

- `get_curr_cluster_cardinality()` : Returns the current number of particles in the central cluster.
- `get_curr_radius_euclidean_length()` : Computes the Euclidean distance (or L2 norm) from the center to the farthest particle.
- `get_curr_radius_manhattan_length()` : Computes the Manhattan distance (or L1 norm) from the center to the farthest particle.
- `get_ring_cluster_cardinality()` : Returns the number of particles within a ring-shaped region around the cluster.
- `get_curr_cluster_sizes()` : Returns a list of sizes of all detected clusters. The length of this list determines the number of clusters in the system at a given time.

The simulation software provides ready-made specialized callable functions, implemented for different experimental needs (visualization, statistical analysis, or computational experiments). Users can select the appropriate function depending on what they require. Below is an overview of the simulation-running functions and their purposes:

- `run_simulation()` : Runs the standard simulation with full graphical rendering in real time. This function allows the user to visually observe the evolution of the particle system.
- `run_simulation_calculations_only()` : Executes the simulation without graphical rendering. This significantly improves computational efficiency and is useful when visualization is not needed.
- `run_simulation_keep_track_of_middle_cluster_size()` : Tracks the evolution of the central cluster size over time. This method records data at predefined intervals, useful for analyzing how clustering progresses.

- `run_simulation_get_iteration_of_first_single_cluster()` : Runs the simulation until the system reaches a single-clustered state for the first time. It returns the iteration number at which this occurs.
- `SIMPLE_run_simulation_get_iteration_of_first_single_cluster()` : A computationally optimized version of the previous method, without additional tracking features, designed for faster execution.
- `run_simulation_get_num_clusters_at_each_iteration()` : Monitors the number of clusters present in the system at each iteration. The results can be used to study the merging dynamics of clusters over time.
- `run_simulation_latest_experiment()` : This function is designed to collect detailed data, such as the time taken to reach a single-clustered state, the number of particles in large clusters, their density relative to the total particles in the system, and more. This function will be useful in data gathering for Section 4

2.3.2 User Controls

The simulation provides interactive keyboard controls for real-time adjustments:

- **Spacebar**: Pause or resume the simulation.
- **F**: Advance the simulation by one iteration (step mode).
- **D**: Increase the simulation speed by updating `self.refresh_rate` to $\min(\text{self.refresh_rate} \times 1.1, 150)$.
- **S**: Decrease the simulation speed by updating `self.refresh_rate` to $\max(\text{self.refresh_rate} \times 0.9, 1)$.
- **R**: Toggle rendering on or off (for performance optimization).
- **I**: Toggle the display of run information, including run ID and iteration, for a clearer view of the simulation.
- **C**: Print the cardinality of the origin cluster.
- **X**: Retrieve the sizes of all detected clusters.

3 Visual Analysis

3.1 Example 2D rendered simulation

The bigger part of the simulations will usually be undertaken only computationally without rendering images in order to optimize the speed and scale up the number of simulations and particles. However, we will choose to render parts of some simulations to visualize and interpret the results of those calculations. The following are screenshots of a typical 100 by 100 simulation rendered at different intervals with the following parameters: $\mu = 0.4$, $\delta = 10^{-4}$, $\epsilon = 0.9$, $\alpha = 0.8$.

First, as expected by construction, the simulation starts at iteration $t = 1$ with particles that are uniformly distributed independently and identically around the torus, as shown in Figure 3.1.



Figure 3.1: Initial state of the simulation with particles uniformly distributed.

In a few iterations (usually around the $t = 50000$ iteration mark), we can start to see particles bumping into and blocking each other, initiating the formation of the first few small-sized clusters as seen in Figure 3.2. More iterations later, as smaller clusters get absorbed by larger clusters, we see fewer but bigger clusters forming (Figure 3.3). This process continues until there is only one cluster left in the entire system.

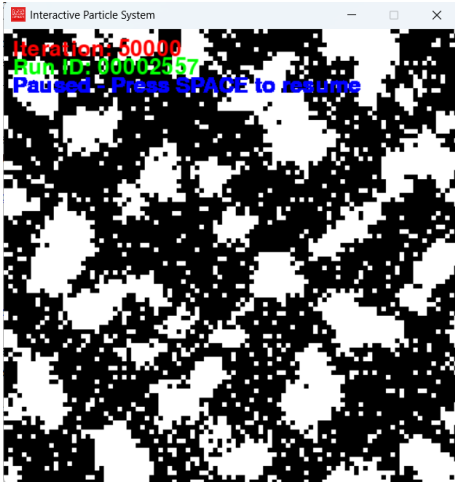


Figure 3.2: Small clusters start forming as particles interact.

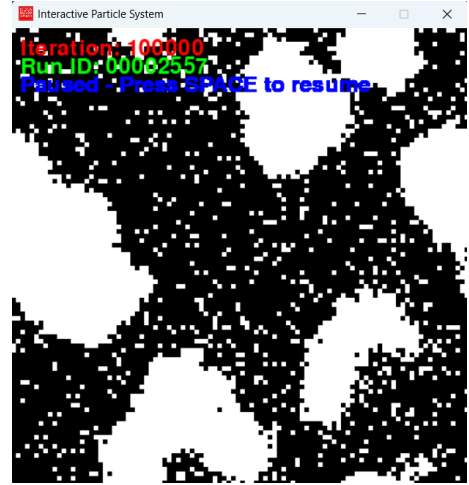


Figure 3.3: Larger clusters emerge as the simulation progresses.

For this example simulation in particular, we observed that by iteration $t = 2800000$ we already had a single-clustered system (Figure 3.4). Usually, the first time a particle system with these parameters becomes single-clustered occurs much sooner, often at $t < 1000000$, as we will see in Section 4

In some cases, our interactive particle system can reach a metastable configuration before eventually grouping into a single cluster, which is likely what happened here. A *metastable clustering state* refers to a temporary arrangement in which clusters appear stable over long timescales but eventually dissolve or reorganize into a more stable state due to stochastic fluctuations or system dynamics [1]. It's somewhat of an illusion of stability, where the cluster appears stationary to the user but is in fact gradually dissolving at an imperceptibly slow rate. As a result, the system needs to run longer before any noticeable changes re-emerges.

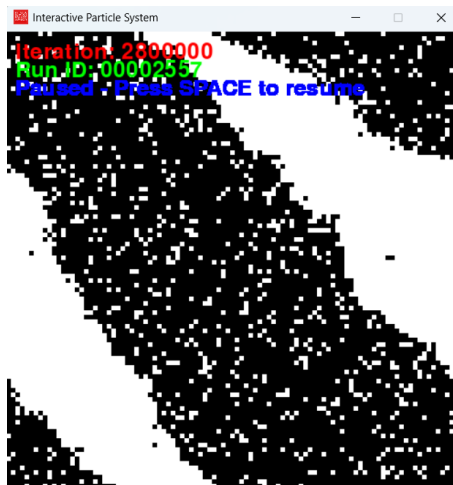


Figure 3.4: Larger clusters emerge as the simulation progresses.

The cluster is not yet in its final form. This occurs only later, as seen in Figure 3.5a where the

particles in the cluster are more consolidated, the way they tend to stay. The system however has not reached a blocking/stable state yet, in fact, it won't, and we will see shortly why.

One might notice that the final cluster looks diagonal, and it is interesting to see that this is indeed a reoccurring theme. Altering the parameters μ , δ , ϵ , α a little might cause the cluster to face the other way (see figures below), however the final cluster will always be diagonal and 45° -angled.

The cluster will stay in this shape but will keep moving in perpetuity around the torus at a slow rate. Indeed, the adjacent Figure 3.5b shows that around 100000000 iteration later the cluster had shifted a little to the right. This was hard to catch at first, and is a good example of why visual rendering of the system is important to better understand it.

Since the system always reaches this state regardless of parameters, it is **translation invariant**.

Definition (Translation Invariance). Let P be the set of all possible configurations of the particle system, and define the translation function

$$\sigma_v : P \rightarrow P, \quad \sigma_v(\eta) = \eta + v, \quad \text{for all } \eta \in P, v \in \mathbb{Z}^2.$$

A particle system with initial configuration η_0 evolves to configuration η_T at time T . The system is said to be translation invariant if, for all $v \in \mathbb{Z}^2$,

$$\sigma_v(\eta_0) = \eta_0 + v \quad \Rightarrow \quad \sigma_v(\eta_T) = \eta_T + v.$$

That is, applying a translation to the initial configuration results in the same translation applied to the configuration at time T . This concept is discussed in Liggett [5].

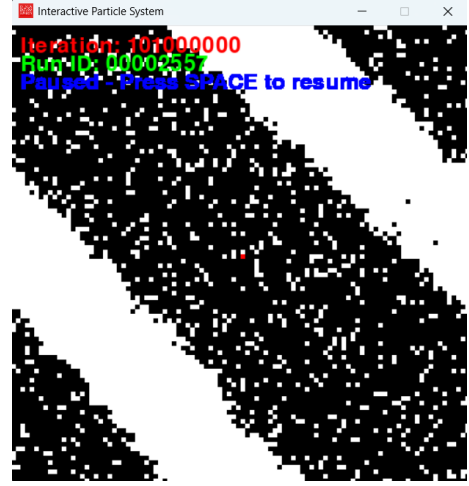
Remarks: - Even if the system eventually forms a unique cluster, this does not mean that it is a *blocking configuration*. The system remains dynamic, allowing the cluster to fluctuate and oscillate within the torus over time.

- The principle of translation invariance states that if we take an initial configuration and its planned evolution, then shift all particles by some vector v in $\mathbb{Z}^2$ modulo the torus, the resulting configuration at time T will match the shifted version of the original configuration at time T .

- **Probabilistic interpretation:** The probability law governing the evolution of the process remains unchanged under translations. That is, shifting the system does not alter its probabilistic behavior or statistical properties.



(a) Stable clustering state.



(b) Stable clustering state prolonged.

Figure 3.5: Comparison of stable clustering states.



Figure 3.6: $\alpha = 1.0$

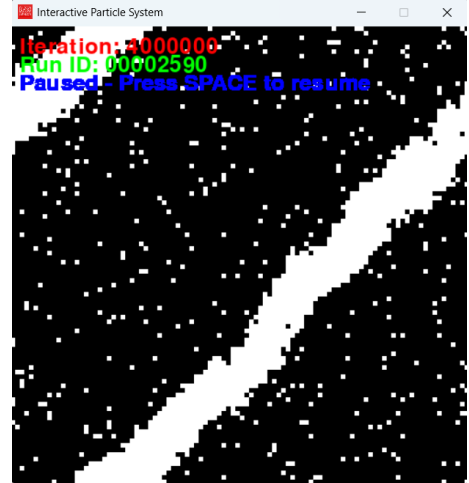


Figure 3.7: $\mu = 0.2$

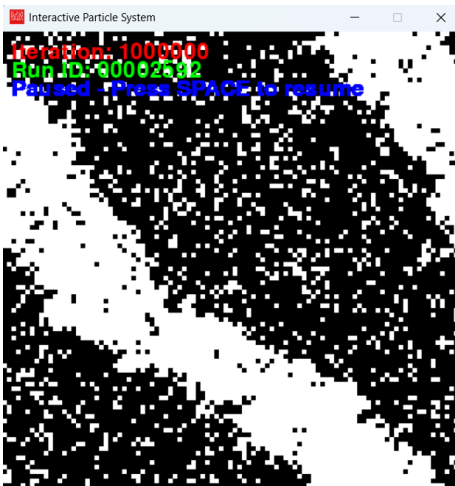


Figure 3.8: $\delta = 10^{-2}$



Figure 3.9: $\epsilon = 0.5$

Interestingly, in Figure 3.6, $\alpha = 1.0$ eliminates the dense–dilute phase coexistence characteristic of motility-induced phase separation: the system condenses fully, leaving virtually no dilute phase.

MIPS, first described by Cates and Tailleur [1], is a nonequilibrium phenomenon where active particles spontaneously separate into dense and dilute phases due to motility-induced clustering, even in the absence of attractive interactions. So far we have seen MIPS in every other combination of parameters except when $\alpha = 1$. In that case, we can count 4 particles visibly not part of the cluster, which suggests that asymptotically, as n tends to infinity, the density of particles blocked or stuck in the big cluster goes to 1. While on the other hand, the other figures suggest, that when $\alpha < 1$, this asymptotic density remains bounded away from 1.

In 3D, this looks like this where the white line is the cluster. Proportions have been exaggerated for the convenience of the reader. For a bird's-eye view, see Abstract page.



Figure 3.10: 3D Torus render.

Initially, when analyzing the diagonality of clusters in 2D, we hypothesized that the emergence of 45° -angled clusters was due to the particles' velocity vectors being uniformly randomly chosen. To test this hypothesis, we modified the distribution of velocity vectors, assigning greater weight to specific directions:

```
DIRECTION_WEIGHTS = [10, # down
                      0,  # right
                      0,  # up
                      1   # left
                      ]

self.v = random.choices(global_possible_directions, weights=self.direction_weights, k=1)[0]
```

Note that the weights do not need to sum to 1, as Python's 'random' package automatically normalizes them.

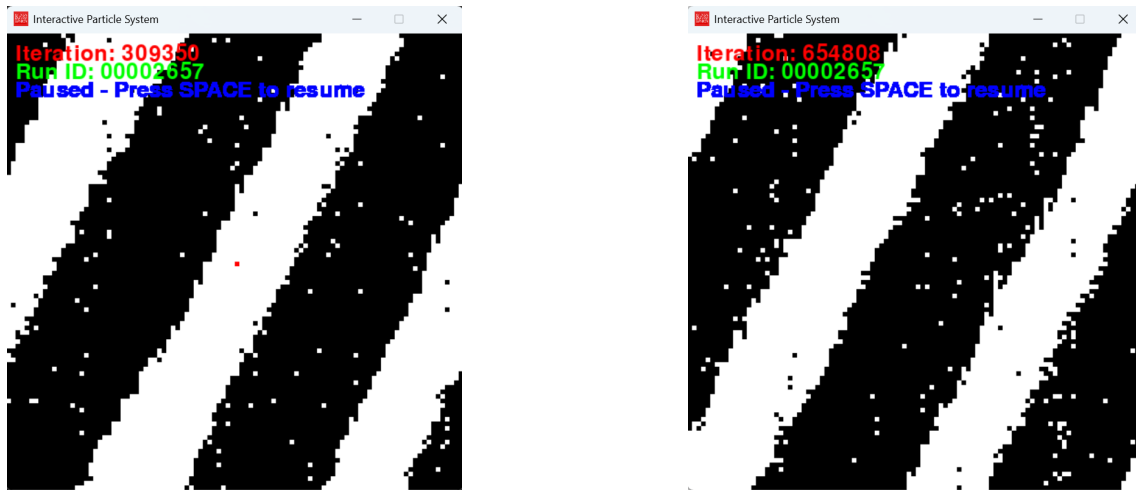


Figure 3.11: Tilted cluster formation under modified directional weights.

As seen in Figure 3.11, the cluster is predominantly downward-facing, with occasional leftward movement due to the small probability of selecting the leftward direction. This minor leftward movement enables particles to eventually interact and form a cluster. However, still images do not fully capture the majestic process of cluster formation.

A useful analogy is to imagine this particle system as *a rainfall of particles with a gentle wind blowing from the right*, nudging particles leftward. Observing the left edge of each "stripe" in the cluster, we notice a **stair-step pattern**, a direct result of particles falling from left-adjacent stripes. This pattern suggests that the cluster can shift leftward relatively quickly, which it does as seen in the adjacent image.

Interestingly, clusters with non-45° angles are relatively rare. We had to experiment extensively with different directional weightings to locate them. In fact, most random weight distributions tend to preserve the default 45° diagonal clustering. To generate a non-45° cluster, the directional weights must follow these conditions: - A **large weight** for movement in one direction. - A **small weight** for a second direction, which **must not** be the opposite of the dominant direction (e.g., if the dominant direction is "up," the second direction cannot be "down"). - The remaining two directions should have weights close to zero.

Without satisfying these constraints, clusters tend to revert to 45° angles over time. This is because even a small probability for a given direction can significantly influence clustering. When a particular direction is strongly favored, particles moving in that direction frequently block each other, limiting movement. Conversely, the direction with a small weight remains relatively unoccupied, allowing unobstructed motion, in the rare occasions that this direction is selected.

For instance, a directional weight vector like:

```
DIRECTION_WEIGHTS = [0.4, # right
```

```

0.4, # down
0.1, # up
0.1 # left
]

```

counterintuitively fails to produce clusters with angles different from 45° . The balanced weights ensure that particles move frequently in all directions, maintaining the typical diagonal cluster structure.

3.2 Example 1D Rendered Simulation

A similar visual analysis can be conducted for our one-dimensional particle system, where, as in the two-dimensional case, the initial state consists of particles uniformly distributed along a circular lattice. This can be observed in Figure 3.12, where the system initially appears mostly yellow, indicating a balanced mix of filled and empty spaces according to our colormap defined in Figure 2.1.

The following is an instance of our one-dimensional interactive particle system simulation with parameters $N = 25000$, $\mu = 0.6$, $\delta = 10^{-3}$, $\epsilon = 0.9$, $\alpha = 0.8$.

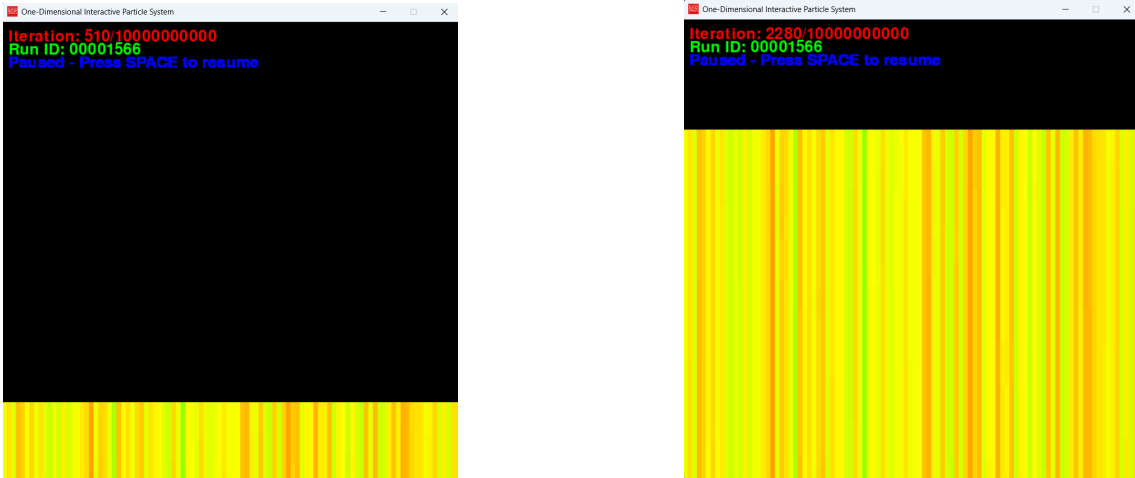


Figure 3.12: Initial configuration of the 1D particle system.

As a reminder, the latest (current) row is located at the bottom of the figure, while all rows above represent past states of the particle system. In the early frames, the upper half of the figure appears black because not enough iterations have passed to fully populate the visualization. As the system evolves, older rows shift upwards to accommodate new data at the bottom.

This should not be confused with the appearance of black squares within a row, which indicate that the space is void of particles (unfortunately not seen here)

Over time, the system transitions from its uniform initial state, with colors shifting towards more

extreme values (red and blue) representing the formation of dense clusters and large voids, respectively. However, unlike the two-dimensional case, we have not observed any stable terminal clusters, even after running the simulation for an extended period.

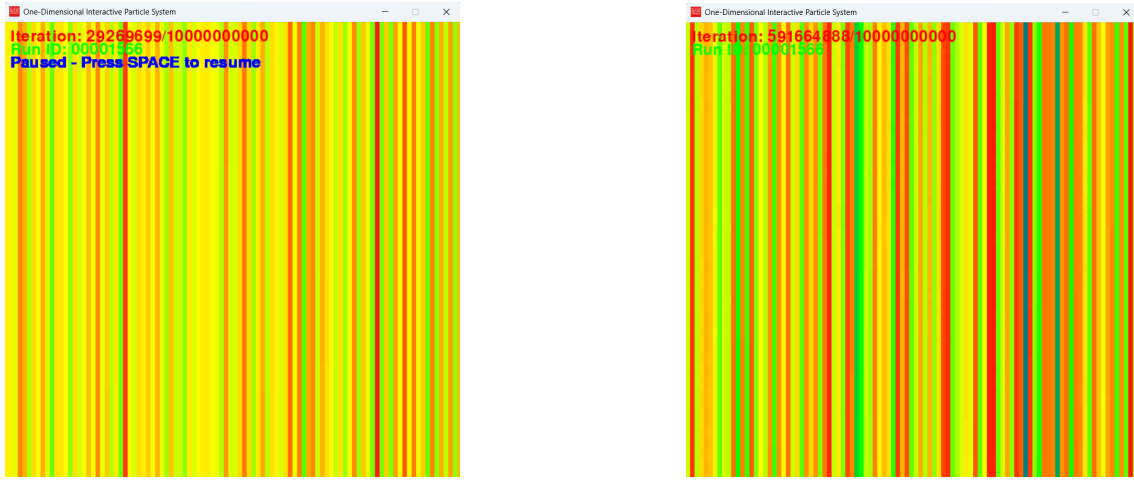


Figure 3.13: Later stages of the 1D interactive particle system.

Despite running the simulation for an extended period (weeks in some cases) no definitive stable clusters have been observed. It remains unclear whether a final, singular cluster would eventually emerge or if the system exhibits some form of persistent motility-induced phase separation. The main limitation in the one-dimensional case is that particles have fewer movement options: they can only move forward or backward along the lattice, making interactions more constrained compared to the two-dimensional system, where particles can move around other blocking particles. This limitation likely explains why clustering progresses so slowly.

Given these observations and the computational cost of indefinitely running the simulation, we chose to focus our in-depth analysis on the two-dimensional case, where clustering behavior is more dynamic and observable within reasonable timescales. As a result, no further calculations or statistical analyses will be presented for the one-dimensional case in the following sections.

4 Statistical Analysis

Understanding the statistical behavior of the particle system is essential for identifying key patterns and trends. This section first examines the impact of different simulation parameters on clustering behavior, using various parameter settings and distribution fitting techniques. Then, we analyze the evolution of the number of clusters over time under different experimental conditions, aiming to characterize how clustering dynamics emerge and evolve in the system.

4.1 Parameter-Wise Impact Analysis

We will now conduct a statistical study to analyze the effects of each parameter on the particle system. To achieve this, we established a baseline set of parameters, corresponding to the first row (Setting 1) in Table 1 to use as benchmark. We then created six additional parameter sets, each differing from the baseline by modifying only one parameter. This allowed us to isolate and examine the impact of individual parameter changes on the system’s behavior.

In an attempt to balance statistical significance and computational feasibility, we decided to conduct 200 simulations per parameter set, each running for 5 million iterations in a 100×100 particle system. These simulations were executed continuously over a period of two weeks, leveraging parallel processing and running without rendering the system to maximize efficiency.

Initially, we considered using a 300×300 world to better approximate large-scale behaviors. However, since computational complexity scales quadratically with system size in our case, this approach quickly became impractical. Even with advanced high-performance computing techniques, such as GPU acceleration and optimized parallel execution, the total runtime would have exceeded feasible limits. Hence, we opted for a 100×100 world, which allowed us to complete all experiments within a reasonable timeframe while still capturing the essential system dynamics.

A key aspect of our analysis is defining a cluster. Without a threshold, and given our cluster size calculation method defined in Section 2.3, even single particles could be misclassified as clusters. To prevent this, we set a minimum cluster size of 25, ensuring only meaningful aggregations are considered.

This choice is based on visual inspection and theoretical expectations. Smaller clusters often appeared transient in renderings, and our previously derived Equation 2.1 supports 25 as a reasonable lower bound given the system parameters.

We observed interesting results in each setting, which we will outline in detail below.

Setting	α	μ	δ	ϵ	Width	Height	T
1	0.8	0.4	0.0001	0.9	100	100	5,000,000
2	1.0	0.4	0.0001	0.9	100	100	5,000,000
3	0.8	0.2	0.0001	0.9	100	100	5,000,000
4	0.8	0.6	0.0001	0.9	100	100	5,000,000
5	0.8	0.4	0.01	0.9	100	100	5,000,000
6	0.8	0.4	0.001	0.9	100	100	5,000,000
7	0.8	0.4	0.0001	0.5	100	100	5,000,000

Minimum Cluster Size: 25

Table 1: Parameter Settings

4.1.1 Baseline Parameters Analysis

First, we examine the performance of the **baseline simulation** (Setting 1). To achieve this, we compiled a standardized report summarizing key statistics of interest. Specifically, we analyze:

1. The iteration at which the particle system first becomes singly clustered.
2. The density of clustered particles versus free particles.
3. A pie chart representing the number of clusters at the end of the simulation and their frequency of occurrence.

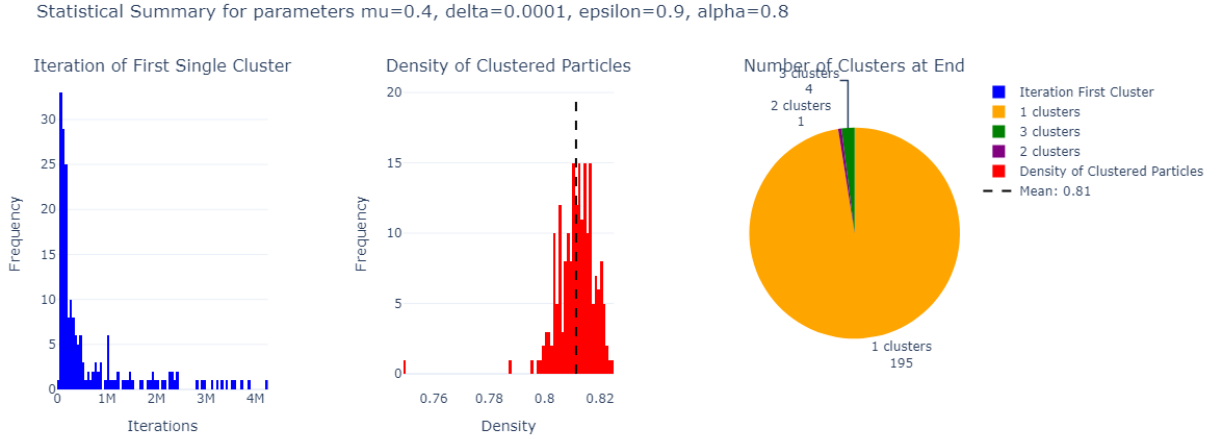


Figure 4.1: Simulation Data Summary Statistics - Setting 1

Interestingly, the first (blue) graph appears to follow a Pareto or Exponential distribution, which aligns with our earlier discussion on metastable configurations delaying cluster formation. Given our interest in this phenomenon, we conducted a separate experiment focused specifically on this setting. In this experiment, we terminated the simulation as soon as the first single cluster formed,

rather than running it for the full 5,000,000 iterations. Since most simulations ended much earlier (as we see already from the blue histogram), they ran significantly faster, allowing us to complete the experiment in just half a day, and running 600 simulations.

To analyze the clustering behavior of our particle system, we attempt to fit the two distributions to our empirical data on the number of iterations required to reach a single cluster. To determine the better fit, we compute:

- Maximum Likelihood Estimates (MLEs) for the parameters,
- Log-likelihood values for model comparison,
- Confidence intervals for the estimated parameters.

First, let's try to fit the Exponential distribution to our data. The probability density function (PDF) of an Exponential distribution is:

$$f(x|\lambda) = \lambda e^{-\lambda x}, \quad x > 0. \quad (4.1)$$

The Maximum Likelihood Estimate (MLE) for λ is given by:

$$\hat{\lambda} = \frac{1}{\bar{x}}, \quad \text{where} \quad \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i. \quad (4.2)$$

Substituting the sample mean from our dataset:

$$\bar{x} = 933251, \quad \text{so} \quad \hat{\lambda} = \frac{1}{933251} \approx 1.0716 \times 10^{-6}. \quad (4.3)$$

To assess the goodness of fit, we compute the log-likelihood function:

$$\log L(\lambda) = n \log \lambda - \lambda \sum_{i=1}^n x_i. \quad (4.4)$$

Substituting $\hat{\lambda}$:

$$\log L(\hat{\lambda}) = n \log \left(\frac{1}{\bar{x}} \right) - n. \quad (4.5)$$

With our computed values:

$$\log L(\hat{\lambda}) = n \log(1.0716 \times 10^{-6}) - n. \quad (4.6)$$

A 95% confidence interval for λ is given by:

$$\left(\hat{\lambda} - z_{\alpha/2} \frac{\hat{\lambda}}{\sqrt{n}}, \quad \hat{\lambda} + z_{\alpha/2} \frac{\hat{\lambda}}{\sqrt{n}} \right). \quad (4.7)$$

Substituting our values:

$$(9.858 \times 10^{-7}, 1.1573 \times 10^{-6}). \quad (4.8)$$

Next, we attempt to fit a Pareto distribution to the data. The Pareto PDF is given by:

$$f(x|\alpha, x_m) = \frac{\alpha x_m^\alpha}{x^{\alpha+1}}, \quad x \geq x_m. \quad (4.9)$$

The Maximum Likelihood Estimate (MLE) for α is:

$$\hat{\alpha} = \frac{n}{\sum_{i=1}^n \log(x_i/x_m)}. \quad (4.10)$$

Substituting our values:

$$\hat{\alpha} = \frac{n}{\sum_{i=1}^n \log(x_i/50001)} \approx 0.482. \quad (4.11)$$

The corresponding log-likelihood function is:

$$\log L(\alpha) = n \log \alpha + n\alpha \log x_m - (\alpha + 1) \sum_{i=1}^n \log x_i. \quad (4.12)$$

Computing with our values:

$$\log L_{\text{Exponential}} = -8847.86, \quad \log L_{\text{Pareto}} = -8772.84. \quad (4.13)$$

Since the Pareto log-likelihood is higher, it provides a better fit to the data.

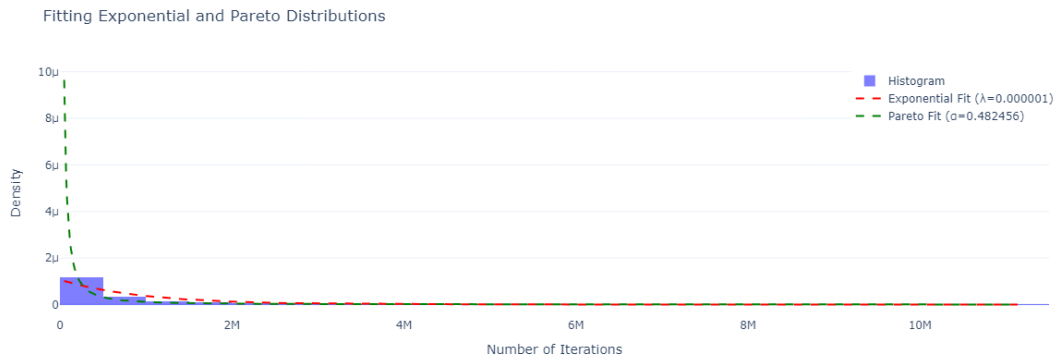


Figure 4.2: Histogram of the number of iterations with fitted Exponential and Pareto distributions

The estimated shape parameter $\hat{\alpha} = 0.482$ suggests a heavy-tailed distribution, meaning that clustering times can be exceptionally large with non-negligible probability. Additionally, the scale parameter $x_m = 50,001$ confirms that no clustering occurs below this threshold of iterations.

This result supports the idea that metastable configurations can sometimes significantly affect the clustering process. The presence of a heavy tail suggests that some simulations remain in metastable states for an extended time before eventually forming a single cluster, and highlights the need to account for metastability in analyzing the system's behavior.

It is worth noting that a Pareto shape parameter $\hat{\alpha} < 1$ implies that the fitted distribution has an *infinite* mean. Consequently, the empirical sample mean $\bar{x}$ should be interpreted with caution under this model: it does not converge to a finite population mean, and summary statistics based on it are dominated by the largest observed waiting times. This is a further manifestation of the heavy-tailed, metastability-driven nature of the clustering times.

A similar statistical analysis can be conducted for the density of clustered particles. Given that the density values lie within the interval $(0, 1)$, we attempt to fit a Beta distribution to the data.

The probability density function (PDF) of a Beta distribution is:

$$f(x|\alpha, \beta) = \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha, \beta)}, \quad 0 < x < 1, \quad (4.14)$$

where α and β are shape parameters, and $B(\alpha, \beta)$ is the Beta function:

$$B(\alpha, \beta) = \int_0^1 t^{\alpha-1}(1-t)^{\beta-1} dt. \quad (4.15)$$

To estimate the parameters, we employ the method of moments. The mean and variance of the Beta distribution are given by:

$$E[X] = \frac{\alpha}{\alpha + \beta}, \quad (4.16)$$

$$\text{Var}(X) = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}. \quad (4.17)$$

From the dataset, we compute the empirical mean and variance:

$$\hat{\mu} = 0.8114, \quad \hat{\sigma}^2 = 5.75 \times 10^{-5}. \quad (4.18)$$

Rearranging the expectation equation, we express β in terms of α :

$$\beta = \alpha \left(\frac{1 - \hat{\mu}}{\hat{\mu}} \right). \quad (4.19)$$

Substituting into the variance equation:

$$\hat{\sigma}^2 = \frac{\alpha(\alpha + \frac{\alpha(1-\hat{\mu})}{\hat{\mu}})}{(\alpha + \frac{\alpha(1-\hat{\mu})}{\hat{\mu}})^2 (\alpha + \frac{\alpha(1-\hat{\mu})}{\hat{\mu}} + 1)}. \quad (4.20)$$

Solving for α and β :

$$\hat{\alpha} = 2158.98, \quad \hat{\beta} = 501.97. \quad (4.21)$$

To further assess the fit, we compute the log-likelihood function:

$$\log L(\alpha, \beta) = (\alpha - 1) \sum_{i=1}^n \log x_i + (\beta - 1) \sum_{i=1}^n \log(1 - x_i) - n \log B(\alpha, \beta). \quad (4.22)$$

Substituting the estimated parameters, we obtain:

$$\log L(\hat{\alpha}, \hat{\beta}) = 697.75. \quad (4.23)$$

The fitted Beta distribution closely aligns with the observed density values, as shown in Figure 4.3. Note that the parameters displayed in the figure legend were obtained by direct numerical maximum-likelihood fitting, which yields slightly different values from the moment-based estimates above; the two fits are visually indistinguishable.

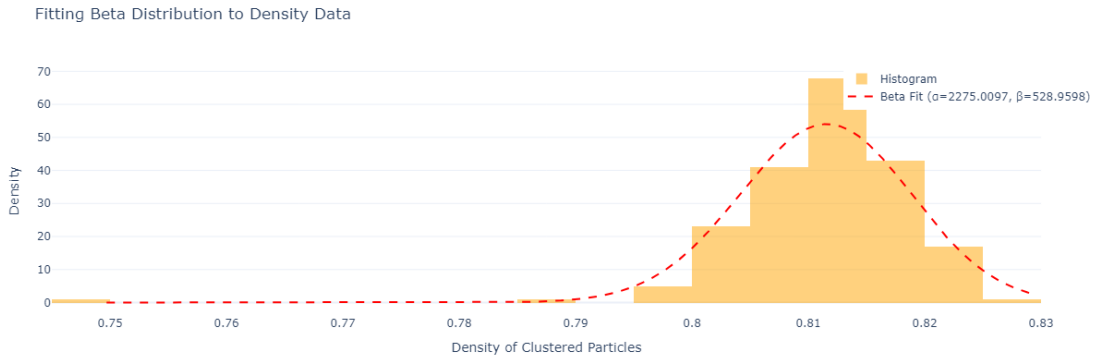


Figure 4.3: Histogram of clustered particle density with fitted Beta distribution

These results indicate that the density of clustered particles follows a Beta distribution with the estimated parameters, supporting the hypothesis that the clustering behavior remains constrained within a stable density range over multiple runs.

4.1.2 Other Settings of Parameters: α

The first parameter change we consider is setting 2 where $\alpha = 1$. As observed earlier, this effectively eliminates motility-induced phase separation (see Figure 3.6). Unlike the baseline parameter set, this configuration exhibits a significantly higher tendency to cluster: all 200 simulations resulted in a singly-clustered particle system, and clustering occurred much earlier, with most simulations reaching a single cluster in fewer than $t = 500,000$ iterations. Additionally, the clusters formed are notably denser, with nearly 100% of the particles in the system belonging to the final cluster: the dilute phase essentially vanishes, so instead of the dense–dilute phase *coexistence* characteristic of MIPS, the system condenses fully into a single dense cluster.

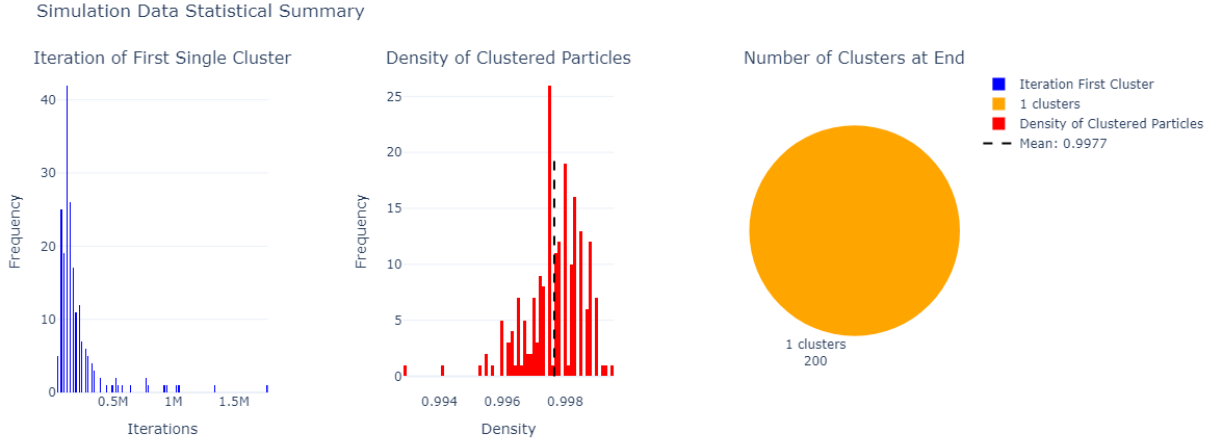


Figure 4.4: Simulation Data Summary Statistics - Setting 2

In fact, as we will see, the absence of inactive particles emerges as the most influential factor in eliminating the coexisting dilute phase. No other parameter combination achieves a density of clustered particles as high as that observed when setting $\alpha = 1$.

This behavior aligns with our previous discussion that the density of clustered particles approaches 1 as n (width and height of the system) tends to infinity.

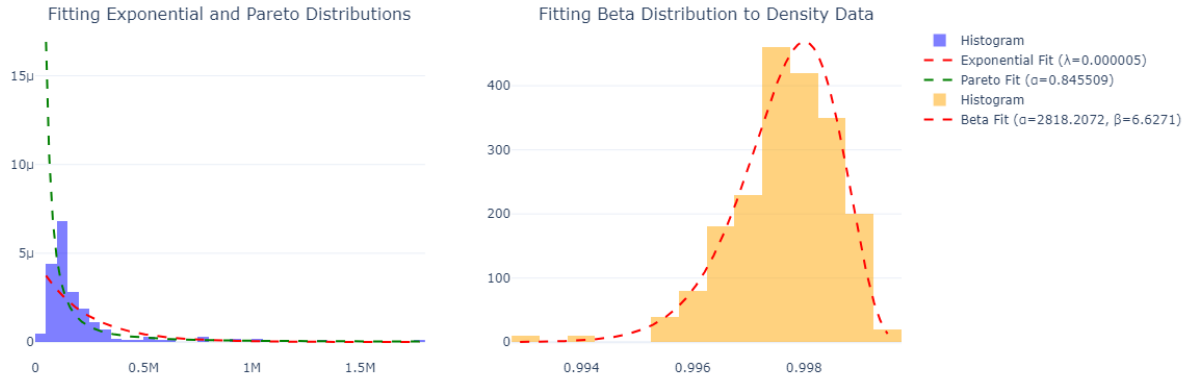


Figure 4.5: Setting 2 ($\alpha = 1$) Distribution Fits

In this case as well, the Pareto distribution provides a better fit than the Exponential distribution for the number of iterations until the first single-cluster formation, with log-likelihood values of -2634.06 and -2652.40 , respectively. The estimated Pareto shape parameter is $\alpha = 0.8455$, with a scale of $x_m = 50000$.

For the density of clustered particles, the Beta distribution is well-fitted with estimated parameters $\alpha = 2684.98$ and $\beta = 6.31$, yielding a log-likelihood of 1126.86 .

4.1.3 μ

Next, we run the same experiments with setting 3 from Table 1 (i.e., changing μ to 0.2).

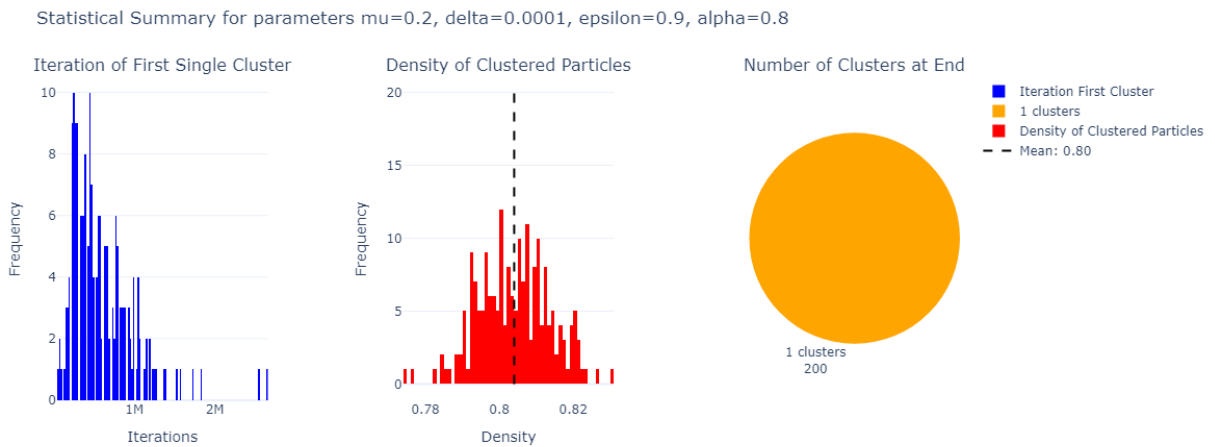


Figure 4.6: Simulation Data Summary Statistics - Setting 3

Again, all systems eventually form a single cluster, and clustering occurs relatively quickly compared to the baseline parameters. However, the mean density of clustered particles remains similar.

Interestingly, in this case, the Exponential distribution provides a better fit than the Pareto distribution for the number of iterations until the first single cluster appears, with log-likelihood values of -2858.34 and -2983.86 , respectively. The estimated Exponential rate parameter is $\lambda \approx 1.69 \times 10^{-6}$, corresponding to a mean clustering time of roughly 590,000 iterations. Since $\mu = 0.2$ leads to a lower density of active particles, clustering events occur more independently, resembling a Poisson process. As a result, the waiting times for the first single cluster formation follow an approximately memoryless process, which is characteristic of the Exponential distribution. This contrasts with the Pareto distribution, which would indicate extreme variations in clustering times that are not observed here.

The Beta fit, with estimated parameters $\alpha = 1326.40$ and $\beta = 323.31$, results in a log-likelihood of 642.43. The relatively high α value compared to β suggests that the distribution is skewed toward higher densities, meaning that most simulations result in a high proportion of clustered particles, though not as dense as in the $\alpha = 1$ case.

However, when we increase μ to 0.6, the best fit remains the Exponential distribution, with a log-likelihood of -2584.34 , compared to -2637.54 for the Pareto distribution. The estimated Exponential rate parameter is $\lambda \approx 6.6 \times 10^{-6}$ (a mean clustering time of roughly 150,000 iterations), reinforcing that the clustering dynamics in this regime continue to exhibit memoryless behavior.

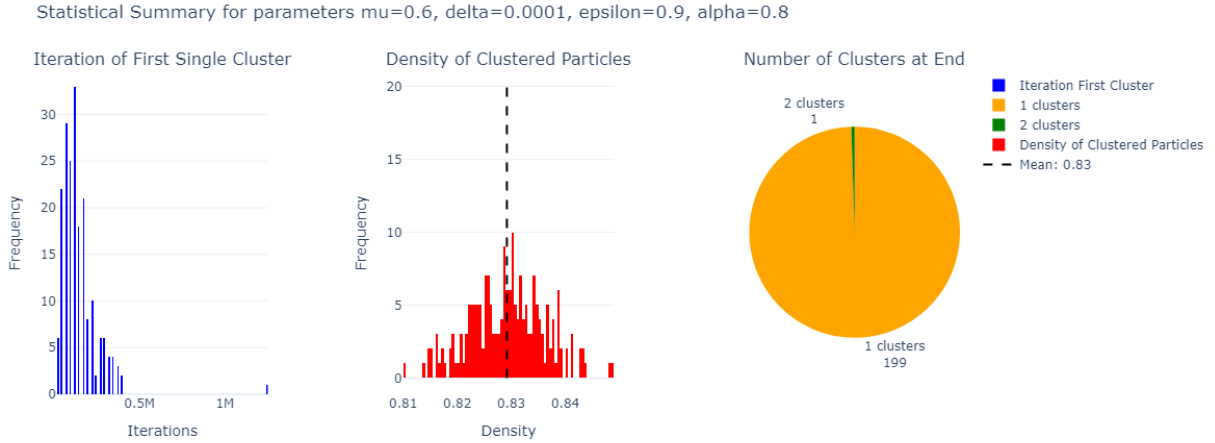


Figure 4.7: Simulation Data Summary Statistics - Setting 4

This suggests that the baseline choice of $\mu = 0.4$ was near an optimal balance for clustering dynamics. We do observe one instance of a system not clustering within 5,000,000 iterations, likely due to the system becoming temporarily trapped in a slow metastable configuration that required additional time to cluster.

For the density of clustered particles, the Beta distribution provides a strong fit, with estimated

parameters $\alpha = 2396.70$ and $\beta = 493.71$, yielding a log-likelihood of 709.00. The relatively high α value compared to β indicates that, while clustering remains efficient, the increased μ results in slightly more variability in the final density of clustered particles.

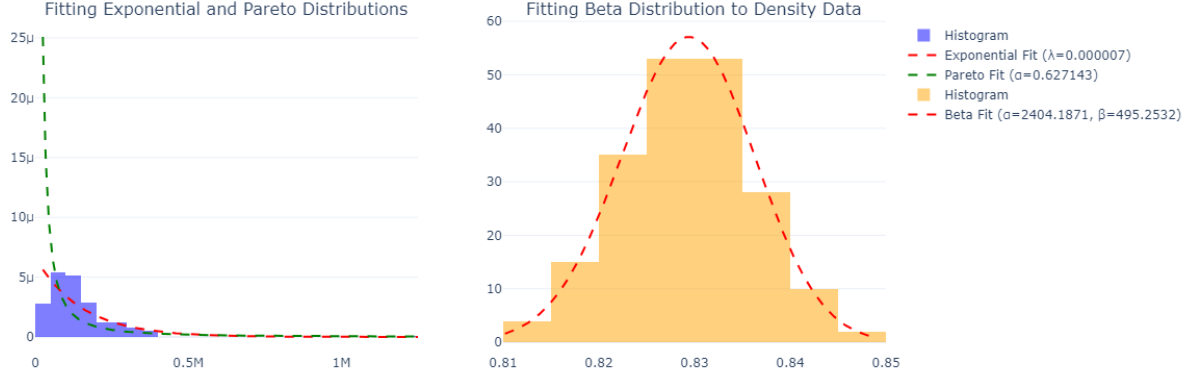


Figure 4.8: Setting 4 ($\mu = 0.6$) Distribution Fits

4.1.4 δ

Next, we examine the case where δ is increased to 0.01, i.e setting 5 in Table 1.

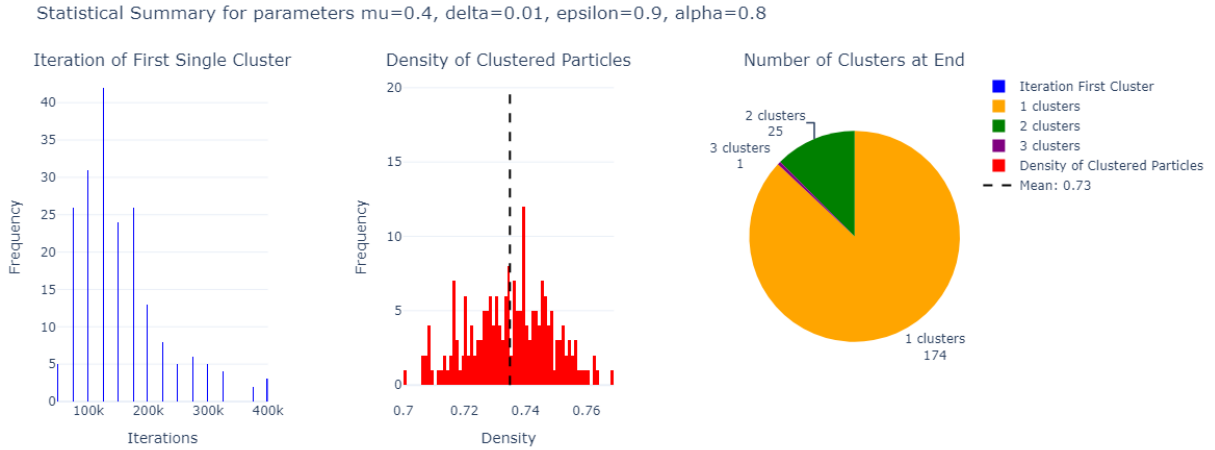


Figure 4.9: Simulation Data Summary Statistics - Setting 5

Increasing δ reduces the persistence of particle movement, making direction changes more frequent and random, which explains the highest number of non-singly clustered systems (26 total) Figure 4.9. Despite this, the system remains highly likely to reach a single-clustered state.

In this case, the Pareto distribution provides a better fit for the number of iterations until the first single cluster appears, with log-likelihood values of -2572.90 and -2588.62 , respectively. The

estimated Pareto shape parameter is $\alpha = 0.9780$, with a scale of $x_m = 50,000$.

A caveat applies here (and, to a lesser extent, wherever a run failed to cluster within the horizon): the fits are computed only on runs that reached a single cluster within $T = 5,000,000$ iterations. The 26 runs that did not are right-censored observations; excluding them systematically underweights the tail of the waiting-time distribution, so the fitted parameters should be regarded as optimistic. A censoring-aware likelihood, treating these runs as contributing $\Pr[X > T]$, would be the principled remedy and is left for future work.

For the density of clustered particles, the Beta distribution provides a good fit, with estimated parameters $\alpha = 776.08$ and $\beta = 279.89$, yielding a log-likelihood of 576.89. The relatively lower α compared to previous settings suggests a slightly broader spread in density values, meaning that the clustering process is slightly less efficient due to the increased rate of direction changes.

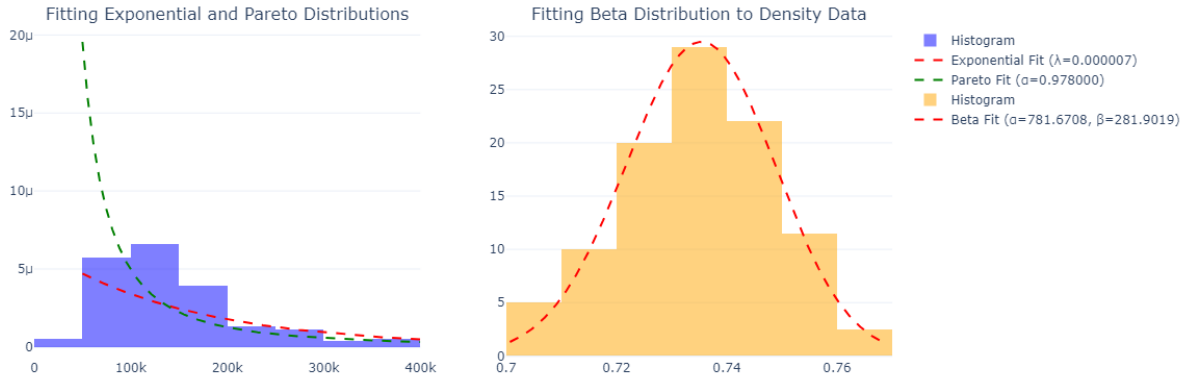


Figure 4.10: Setting 5 ($\delta = 0.01$) Distribution Fits

Next, δ to 0.001, (Setting 6).

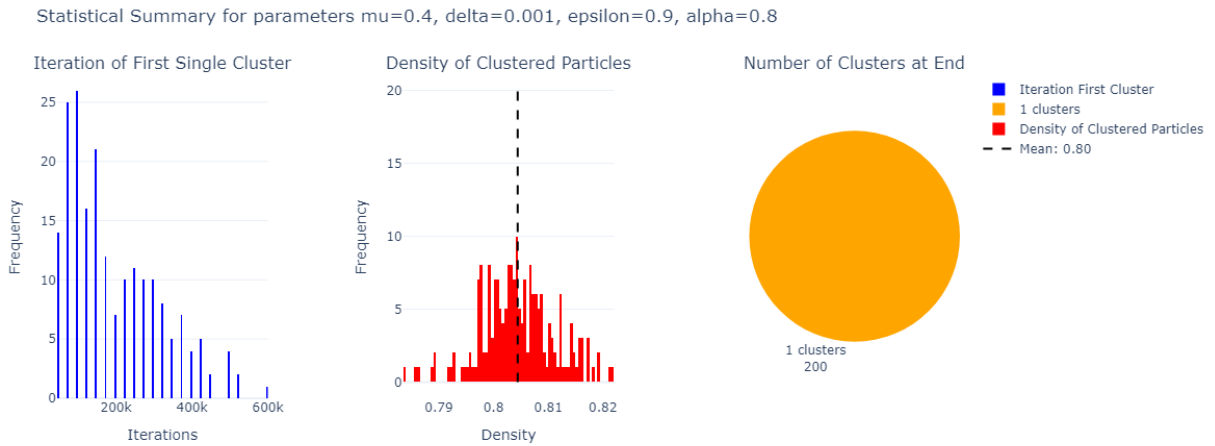


Figure 4.11: Simulation Data Summary Statistics - Setting 6

In contrast to Setting 5, where a higher δ increased the frequency of direction changes and resulted in a slightly lower clustering efficiency, reducing δ to 0.001 re-enhances directional persistence. This leads to a more structured movement pattern, which facilitates the formation of a single cluster more consistently. Indeed, all 200 simulations in this setting resulted in a singly clustered state.

Again, the Pareto distribution provides a better fit for the number of iterations until the first single cluster appears, with log-likelihood values of -2634.82 for the Pareto and -2640.34 for the Exponential fit. The estimated Pareto shape parameter is $\alpha = 0.8440$, with a scale of $x_m = 50,000$. The improved Pareto fit suggests that longer clustering times become more probable due to, again, occasional metastable configurations, though overall clustering remains highly likely.

For the density of clustered particles, the Beta distribution fits well with estimated parameters $\alpha = 2770.01$ and $\beta = 673.48$, yielding a log-likelihood of 716.14. The relatively high α suggests that most simulations result in consistently high clustering densities, similar to previous settings but with slightly less variation compared to $\delta = 0.01$.

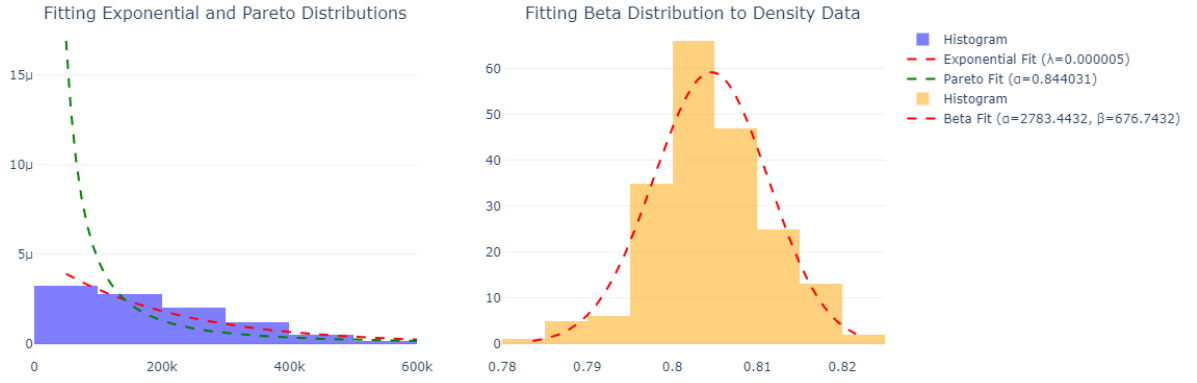


Figure 4.12: Setting 6 ($\delta = 0.001$) Distribution Fits

4.1.5 ϵ

Finally, we consider setting 7, where ϵ is reduced to 0.5.

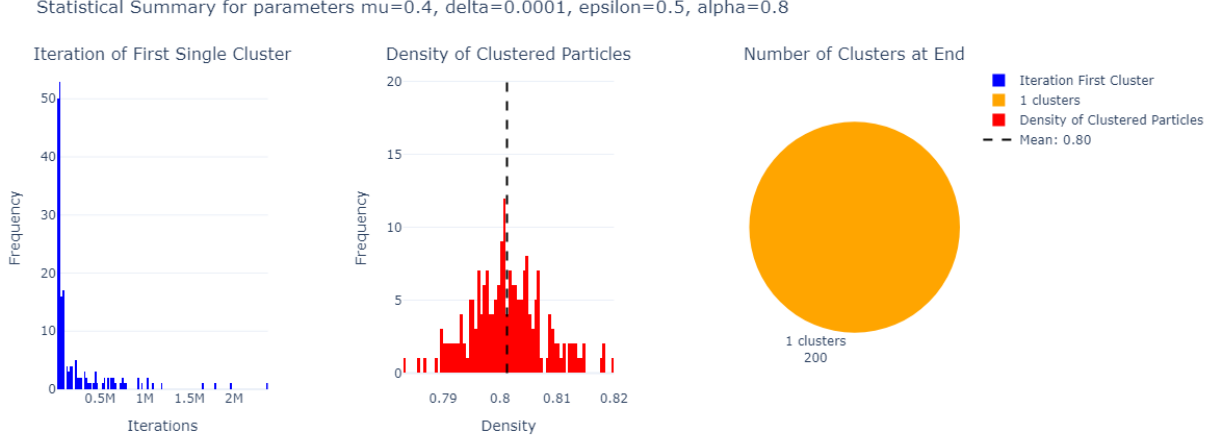


Figure 4.13: Simulation Data Summary Statistics - Setting 7

This modification affects the interaction strength between particles, reducing their alignment probability. Despite this, all simulations still ultimately form a single cluster, and the overall clustering behavior remains largely similar to the baseline case. Similarly, the mean density of clustered particles roughly stays the same.

This time it's easier to visualize that the Pareto distribution provides the best fit for the number of iterations until the first single cluster appears, with log-likelihood values of -2531.26 for the Pareto distribution and -2648.82 for the Exponential distribution. The estimated Pareto shape parameter is $\alpha = 0.7803$, with a scale of $x_m = 50,000$.

For the density of clustered particles, the Beta distribution provides a strong fit, with estimated parameters $\alpha = 3010.78$ and $\beta = 746.94$, yielding a log-likelihood of 723.18 . A high α value relative to β suggests that the majority of simulations result in high-density clusters, but the reduced alignment probability introduces slightly greater variability in the final clustered density compared to the baseline case.

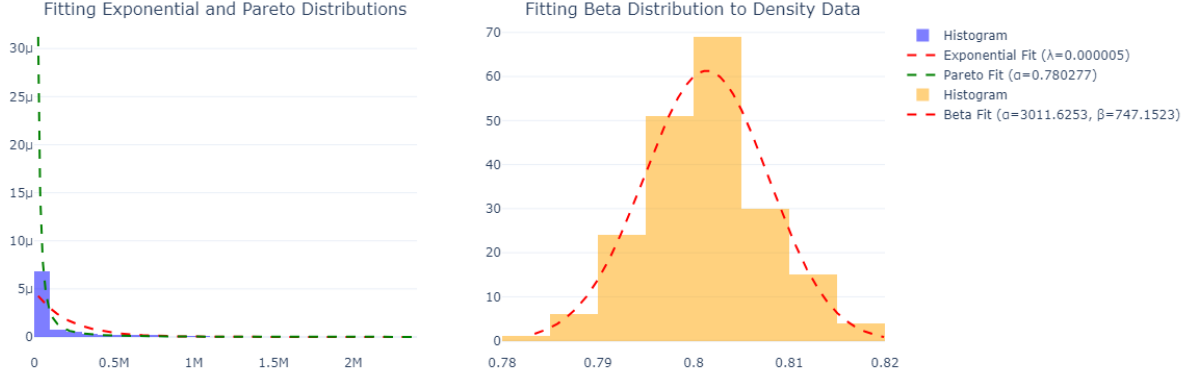


Figure 4.14: Setting 7 ($\epsilon = 0.5$) Distribution Fits

Table 2 summarizes the best-fitting distributions and key parameters for each setting, highlighting how changes in system dynamics affect clustering behavior.

Setting	Changed Parameter	Best Fit	Shape Parameters
1	None	Pareto	$\alpha = 0.482, x_m = 50000$, Beta ($\alpha = 2158.98, \beta = 501.97$)
2	$\alpha = 1$	Pareto	$\alpha = 0.8455, x_m = 50000$, Beta ($\alpha = 2684.98, \beta = 6.31$)
3	$\mu = 0.2$	Exponential	$\lambda \approx 1.69 \times 10^{-6}$, Beta ($\alpha = 1326.40, \beta = 323.31$)
4	$\mu = 0.6$	Exponential	$\lambda \approx 6.6 \times 10^{-6}$, Beta ($\alpha = 2396.70, \beta = 493.71$)
5	$\delta = 0.01$	Pareto	$\alpha = 0.9780, x_m = 50000$, Beta ($\alpha = 776.08, \beta = 279.89$)
6	$\delta = 0.001$	Pareto	$\alpha = 0.8440, x_m = 50000$, Beta ($\alpha = 2770.01, \beta = 673.48$)
7	$\epsilon = 0.5$	Pareto	$\alpha = 0.7803, x_m = 50000$, Beta ($\alpha = 3010.78, \beta = 746.94$)

Table 2: Summary of Distribution Fits and Clustering Behavior for Each Parameter Setting

4.2 Evolution of Total Clusters

Another interesting aspect of our particle system is the evolution of the number of clusters over time. Analyzing this progression provides insight into how quickly the system transitions from a dispersed state to a fully clustered one. Tracking the number of clusters at different time steps allows us to observe patterns such as the rate of cluster merging, the formation of metastable

states, and whether the clustering process follows a rapid or gradual trajectory.

To isolate the effect of system size on clustering dynamics, we use the baseline parameters from the previous section and vary only the width and height of the system (thus also varying the total number of particles) while keeping them equal to each other. This enables us to examine how spatial constraints influence the speed and nature of cluster formation. The table below summarizes the parameter settings for this experiment.

Setting	α	μ	δ	ϵ	Width	Height	T	Minimum Cluster Size
1	0.8	0.4	0.0001	0.9	200	200	1,000,000	25
2	0.8	0.4	0.0001	0.9	100	100	1,000,000	25

Table 3: Parameter settings for studying the evolution of the number of clusters.

To gather meaningful statistics, we ran 50 simulations for each setting, tracking the number of clusters in each system every 25,000 iterations for 1,000,000 iterations. This allows us to construct a detailed picture of how clustering progresses over time and whether larger systems exhibit qualitatively different behavior.

In the following sections, we analyze how the total number of clusters evolves under these conditions, beginning with the larger particle system.

4.2.1 200×200 Particle Systems

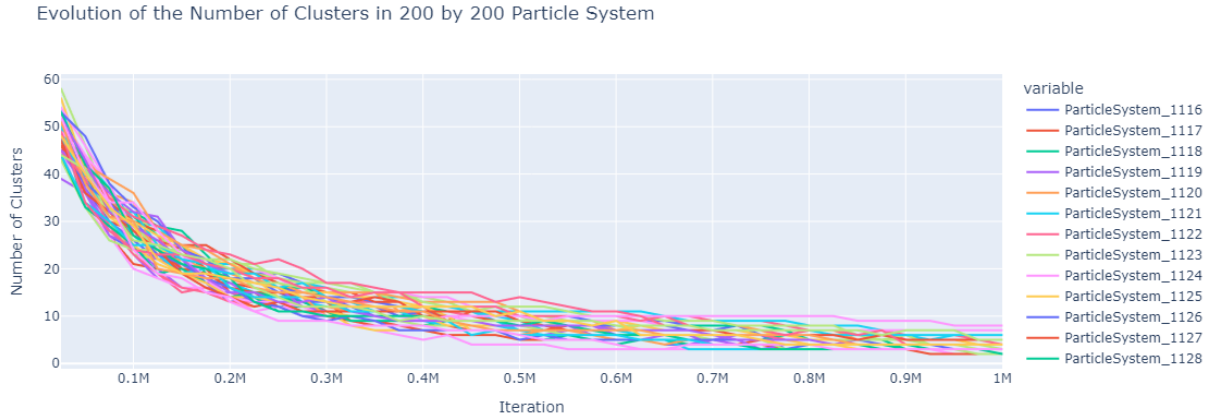


Figure 4.15: Evolution of the Total Number of Clusters - Setting 1

Figure 4.15 shows the evolution of the number of clusters over time in a 200×200 particle system. Each trajectory represents an independent simulation, and we observe a consistent downward trend in the number of clusters as the system evolves and clusters merge, as expected.

Initially, the clustering rate is rapid, with smaller clusters readily merging upon interaction. How-

ever, as the number of clusters decreases, the merging process slows down, making the curve concave. In other words, the transition from n clusters to $n - 1$ clusters takes progressively longer as n decreases. This is intuitive, as early-stage clusters are smaller, containing fewer particles, and thus disintegrate and merge more quickly than larger, denser clusters that form later in the simulation.

The presence of metastable configurations further supports this observation, particularly for iterations $t > 500,000$, where clusters can remain separated for extended periods before eventually merging. This behavior aligns with the expected dynamics of active particle systems.

To quantify the evolution of the number of clusters over time, we perform a quadratic regression of the form:

$$y = ax^2 + bx + c \quad (4.24)$$

where:

- y is the predicted number of clusters,
- x is the iteration count,
- a, b, c are the regression coefficients to be determined.

Given our dataset of iterations x_i and corresponding mean cluster counts y_i , we determine the coefficients a, b, c by minimizing the sum of squared residuals:

$$\sum_{i=1}^n (y_i - (ax_i^2 + bx_i + c))^2 \quad (4.25)$$

This leads to the following system of normal equations:

$$nc + \sum x_i b + \sum x_i^2 a = \sum y_i \quad (4.26)$$

$$c \sum x_i + b \sum x_i^2 + a \sum x_i^3 = \sum x_i y_i \quad (4.27)$$

$$c \sum x_i^2 + b \sum x_i^3 + a \sum x_i^4 = \sum x_i^2 y_i \quad (4.28)$$

Solving this system with the dataset gives the estimated coefficients:

$$a = 6.127168 \times 10^{-11} \quad (4.29)$$

$$b = -9.131397 \times 10^{-5} \quad (4.30)$$

$$c = 37.64621 \quad (4.31)$$

Thus, the final quadratic regression equation is:

$$y = (6.127168 \times 10^{-11})x^2 + (-9.131397 \times 10^{-5})x + 37.64621 \quad (4.32)$$

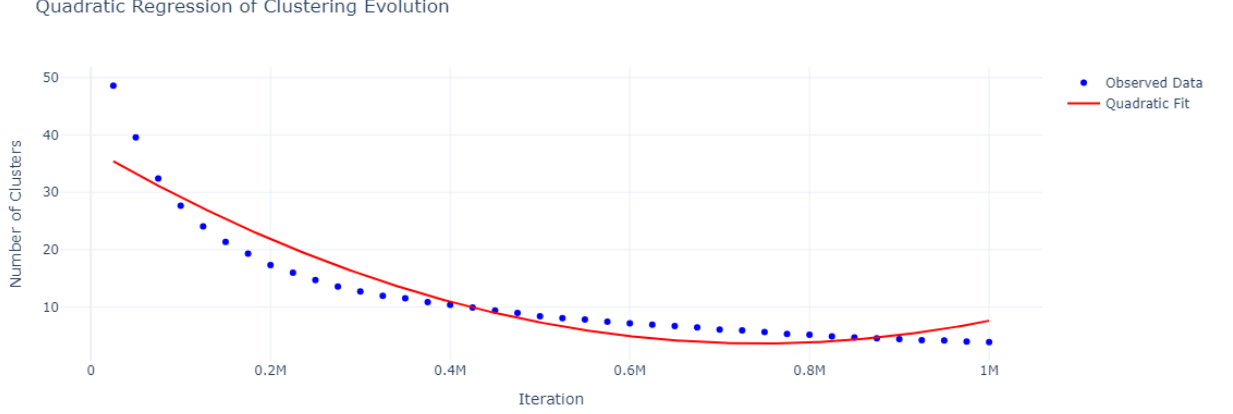


Figure 4.16: Quadratic Regression Fit for Clustering Evolution

The positive quadratic coefficient a captures the decelerating decline in the number of clusters over time, which aligns with the concave shape observed in Figure 4.15, while the negative linear coefficient b reflects the overall downward trend.

An important limitation of this model, however, is visible in Figure 4.16: a parabola with $a > 0$ attains its minimum at $x = -b/2a \approx 745,000$ iterations, *inside* the fitted range, beyond which the fitted curve turns upward while the data continue to decrease monotonically. This upturn is an artifact of the quadratic functional form rather than a feature of the dynamics — the number of clusters cannot increase on average, since clusters only merge. The quadratic fit should therefore be read purely as a local descriptive summary of the early decline; a monotone model such as an exponential decay $y = c_0 e^{-kx} + c_1$ or a power law would be a more faithful description of the full trajectory, and we leave such a fit for future work.

4.2.2 100×100 Particle Systems

In contrast to the larger system, the evolution of the number of clusters in the 100×100 particle system appears more erratic at first glance. However, this is largely due to the smaller scale of the y -axis rather than an actual increase in fluctuations. In reality, the number of clusters in this system varies within a narrower range, typically between 1 and 5, compared to the 200×200 case where values spanned from 2 to 9.

A notable feature in this system is the presence of metastable configurations, exemplified by the red trajectory, where the number of clusters remains constant for an extended period before eventually

decreasing. This suggests that, in smaller systems, clusters may become temporarily stabilized, requiring additional time to merge.

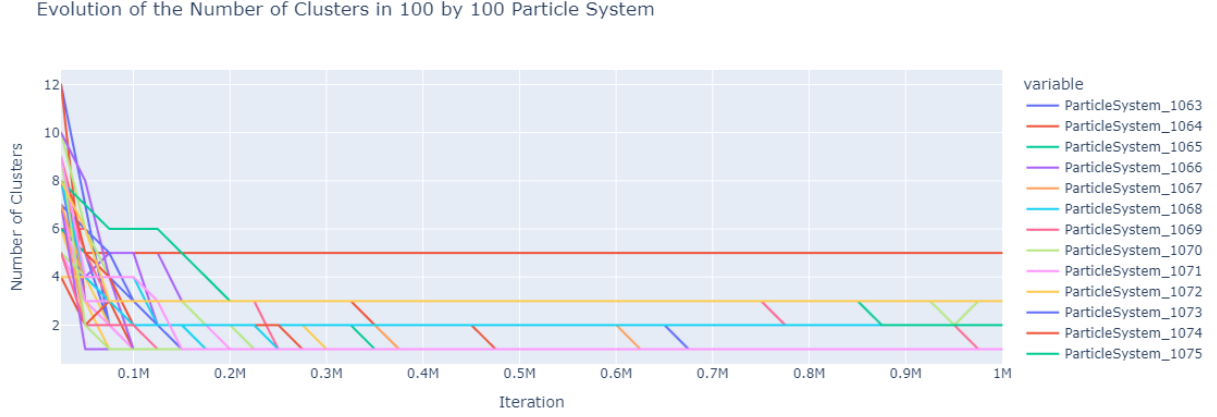


Figure 4.17: Evolution of the Total Number of Clusters - Setting 2

To quantify the overall clustering trend, we again fit a quadratic regression to the data, yielding the following equation:

$$y = (6.46 \times 10^{-12})x^2 + (-8.40 \times 10^{-6})x + 4.04 \quad (4.33)$$

The same caveat as in the 200×200 case applies: this parabola's minimum lies at $x \approx 650,000$ iterations, so the fit is again only a local descriptive summary of the declining phase. While metastable states still occur, the clustering process remains efficient, with the system clustering relatively quickly.

5 Conclusion

This project investigated the clustering dynamics of an active particle system under various parameter settings. Our simulations provided insight into the conditions that promote or inhibit clustering, the timescales involved, and the statistical distributions governing these transitions. The results confirmed that the Pareto distribution generally provides the best fit for the waiting times until full system clustering, except in cases where the system exhibits near-memoryless behavior, favoring the Exponential distribution. Meanwhile, the Beta distribution effectively captured the density of clustered particles across different parameter regimes.

One of the key challenges was the computational cost of running large-scale simulations. The development and optimization of the simulation software required significant time, limiting the opportunity to fully explore the mathematical aspects of the system. Additionally, the vast number of potential directions led to an ambitious but fragmented exploration, leaving several promising avenues for future study.

A notable direction for further research is the study of cluster radii beyond simple cardinality. While much of this work focused on counting clusters and analyzing their formation, a more geometric approach, quantifying cluster size using L1 (Manhattan) and L2 (Euclidean) norms, could yield deeper insights into the spatial structure of clusters. The introduction of ring clusters addressed cases where the system did not cluster precisely at the predefined origin but instead formed a ring around it. Similarly, the experiments with an initially predefined middle cluster explored the stability and evolution of large pre-existing clusters, raising further questions about metastability and possible phase transitions.

Beyond theoretical interest, this work is highly relevant to real-world systems. The behavior of self-propelled particle systems is fundamental to active matter physics, with applications in biological swarms, traffic flow models, hydrodynamics, and percolation theory. The ability to predict clustering behavior and metastable states is crucial for understanding emergent collective phenomena in these fields.

Moving forward, future work could prioritize either the mathematical formalization of clustering behavior or the computational optimization of simulations to scale to even larger systems. Exploring different spatial constraints, boundary conditions, and stochastic movement rules could further enrich our understanding of the fundamental principles governing self-organized clustering.

This study represents only a first step in a vast and rich research area. With improved simulation tools and refined theoretical models, much remains to be uncovered about the fascinating world of active particle systems.

Acknowledgements

I would like to thank my supervisor, Prof. Alexandre Stauffer, for his guidance and support throughout this project! His expertise has been invaluable in shaping my understanding of the subject.

I am also grateful to King's College London and the Department of Mathematics for providing the best academic environment and resources that made this project possible. My time here has been incredibly rewarding and I have greatly enjoyed the challenges and opportunities that came with it.

Finally, I would like to thank my family and friends for their support. As I complete my studies here at King's, I look back on my time at King's with appreciation and fond memories.

Appendix

The simulation software has been entirely built by myself. The classes, methods and other Python files amount to thousands of lines of code and cannot be put in this appendix. For that, I have made them publicly available for any researcher on this: [GitHub Repository](#).

For convenience, it is also pasted here: <https://github.com/MichelFaloughi/bachelor-project>.

To access and download the simulation software, follow one of these two methods:

1. Clone the Repository (Recommended Method)

Assuming you have Git installed and configured on your machine, open a terminal and run:

```
git clone https://github.com/MichelFaloughi/bachelor-project.git
```

This will create a local copy of the repository on your computer.

2. Download as a ZIP File

If you prefer not to use Git, you can download the code manually:

- Go to [GitHub Repository](#) (same link as above)
- Click on the green **"Code"** button.
- Select **"Download ZIP"** and extract the files.

Once downloaded, navigate to the extracted folder or cloned directory. For information on how to use the software, refer to Section 2.3

References

- [1] Michael E. Cates and Julien Tailleur. “Motility-Induced Phase Separation”. In: *Annual Review of Condensed Matter Physics* 6 (2015), pp. 219–244. DOI: 10.1146/annurev-conmatphys-031214-014710.
- [2] Stefan Grosskinsky. *A Course in Interacting Particle Systems*. Lecture notes. 2017.
- [3] Hridayesh Kedia, Shunhao Oh, and Dana Randall. “Local stochastic algorithms for alignment in self-organizing particle systems”. In: *arXiv preprint arXiv:2207.07956* (2022). URL: <https://arxiv.org/abs/2207.07956>.
- [4] Deyue Li. “Hydrodynamic limit for an active-passive exclusion process”. In: *arXiv preprint arXiv:2501.15862* (2025). URL: <https://arxiv.org/abs/2501.15862>.
- [5] T. M. Liggett. “Interacting Particle Systems”. In: *Springer-Verlag* (1985).
- [6] Ubaldo Marini Bettolo Marconi et al. “Dynamical patterns in active-passive particle mixtures with non-reciprocal interactions: exact hydrodynamic analysis”. In: *Physical Review Research* 5.1 (2023), p. 013088. DOI: 10.1103/PhysRevResearch.5.013088. URL: <https://journals.aps.org/prresearch/abstract/10.1103/PhysRevResearch.5.013088>.
- [7] James Mason et al. “Exact hydrodynamics and onset of phase separation for an active exclusion process”. In: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 479.2279 (2023), p. 20230524. DOI: 10.1098/rspa.2023.0524. URL: <https://doi.org/10.1098/rspa.2023.0524>.
- [8] Liheng Yao and Robert L. Jack. “Interfacial and density fluctuations in a lattice model of motility-induced phase separation”. In: *The Journal of Chemical Physics* 162.11 (2024), p. 114902. DOI: 10.1063/5.0139994. URL: <https://doi.org/10.1063/5.0139994>.